Experiment 5: Integrate Java Project with Jenkins

Aim:

To create a Java program and automate build via Jenkins.

Procedure:

- The application should have some basic functionality, such as printing "Hello World" or performing simplecalculations.

Create a simple java program with file name as Demo.java and save it in a folder on desktop

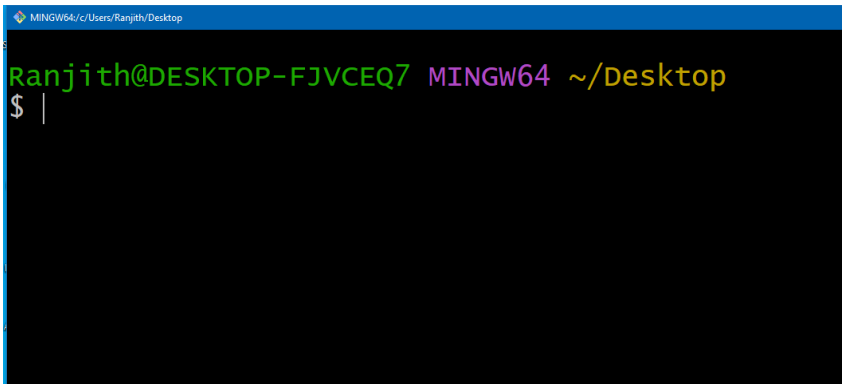
```
public class Demo {

  public static void main (String args[])
  {
  System.out.println("hello world");
  }
  }
```

Commit the code to a Git repository:

- Create a Git repository for the application and commit the code to the repository. Preform the following steps
  1. Login into GitHub account and create a new public repository
  2. After login, click on repositories and click on New button to create the repository
  3. Right click on the desktop and select open Gitbash here

     Gitbash window opens



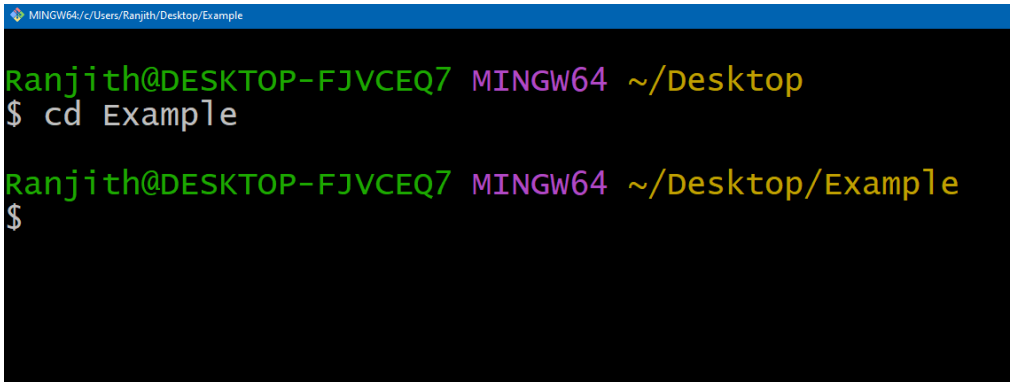
```
MINGW64:/c/Users/Ranjith/Desktop/Example

Ranjith@DESKTOP-FJVCEQ7 MINGW64 ~/Desktop
$ cd Example

Ranjith@DESKTOP-FJVCEQ7 MINGW64 ~/Desktop/Example
$ git init
Initialized empty Git repository in C:/Users/Ranjith/Desktop
Example/.git/

Ranjith@DESKTOP-FJVCEQ7 MINGW64 ~/Desktop/Example (master)
$
```

- Now type the command **cd folder_name** in the above terminal and press enter



- Execute the following commands to create a local repository and push it on to the GitHub remote repository

i) ***git init***

ii) **git config --global user.name "username of githhub account"**

iii) **git config --global user.email "email id provided github account"**

iv) **git add .**

v) **git commit -m "adding files updated"**

```
Ranjith@DESKTOP-FJVCEQ7 MINGW64 ~/Desktop/Example (master)
$ git add .

Ranjith@DESKTOP-FJVCEQ7 MINGW64 ~/Desktop/Example (master)
$ git commit -m "adding file updated"
[master (root-commit) 72471d6] adding file updated
 1 file changed, 7 insertions(+)
 create mode 100644 Demo.java
```

vi) **git remote add originurl address of the git hub repository**

Ex: git remote add origin https://github.com/username/repository.git

Copy the url address of the git repository from git hub and paste in the above command

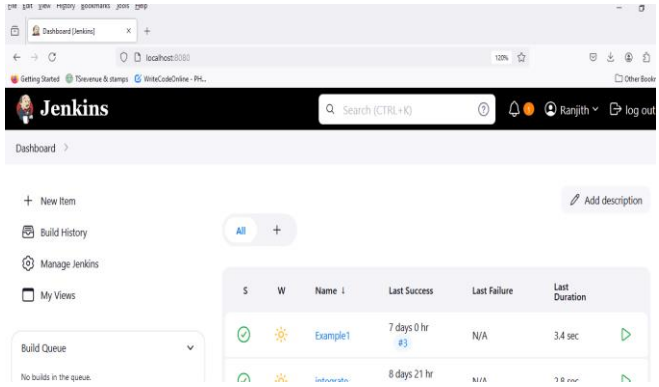
vii) git push -u origin master

Refresh the git repository in GitHub and check that files on local machine are pushed on to git remote repository

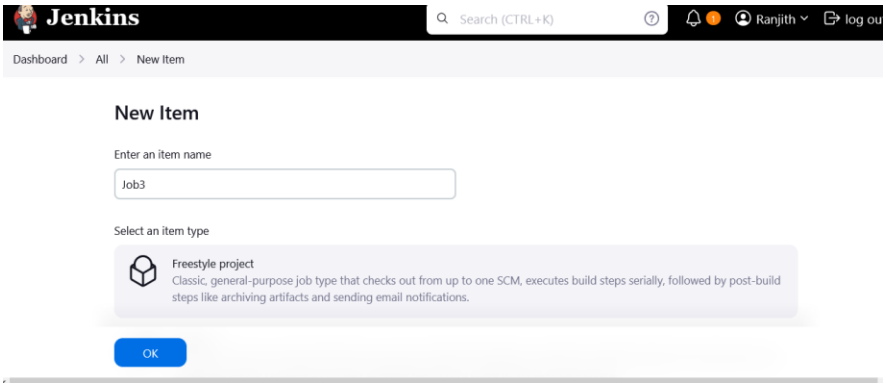
- Make sure that the Git repository is accessible from the Jenkinsserver.

Create a Jenkins job:

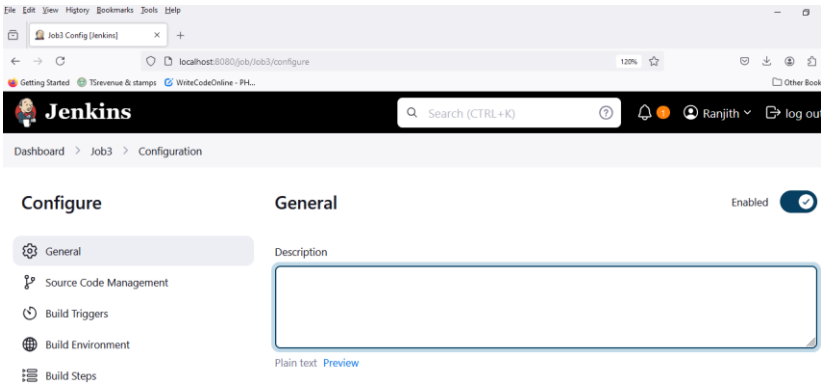
- **Log in to the Jenkins web interface and create a new job.**
- **Configure the job to build the Java application from the Gitrepository.**
- **Specify the build triggers, such as building after every commit to the repository.**



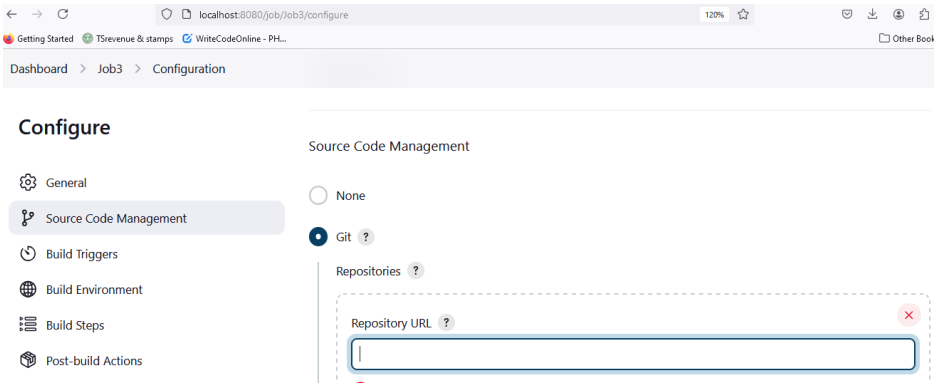
Click on New Item to create a new job



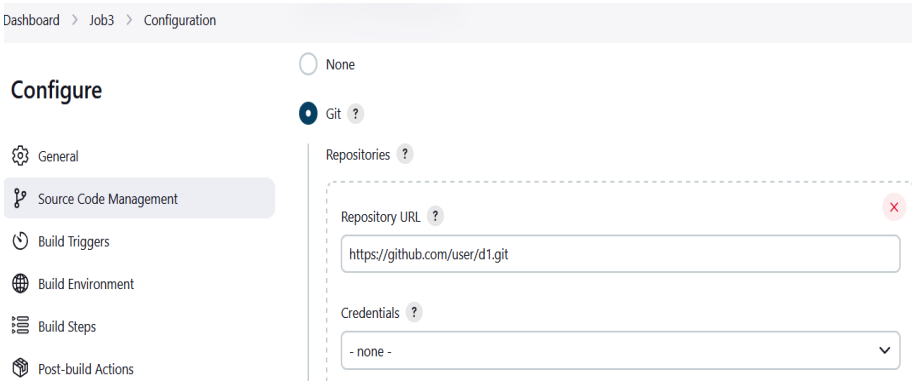
Give the **Item name**, select the item type as***Freestyle project*** and click **OK button**



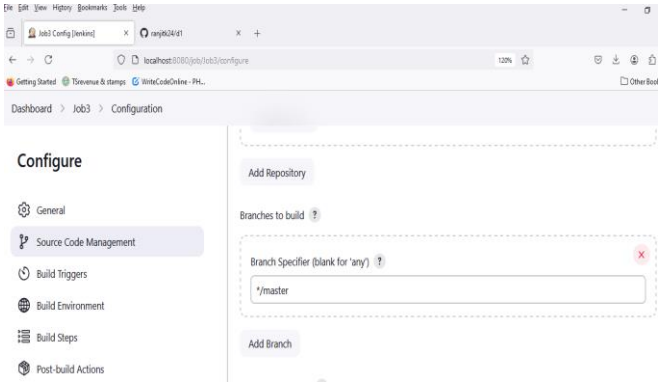
Give the Description of the project or program

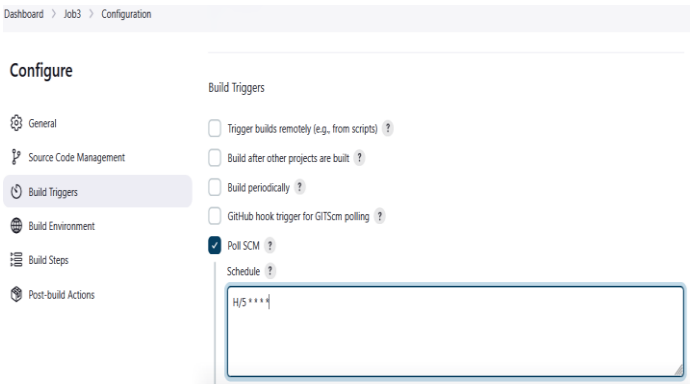


In the source code management, select the option as Git and copy the URL address of the Git repository and paste in Repository URL field

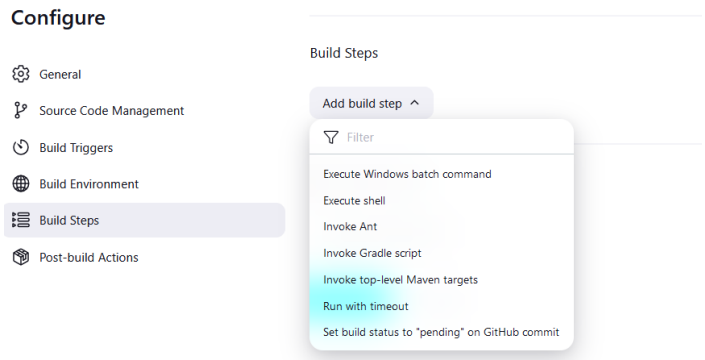


In branches to build option, mention branch specifier as master

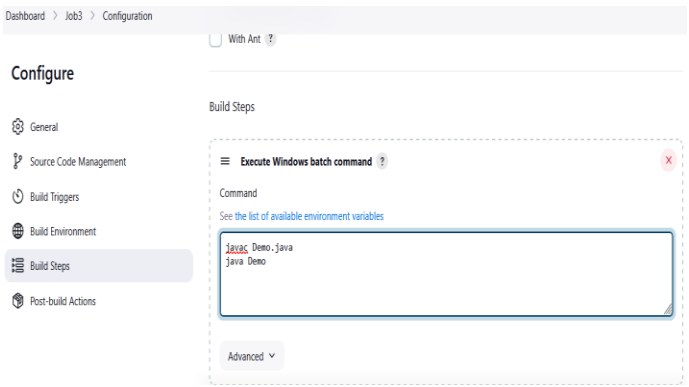




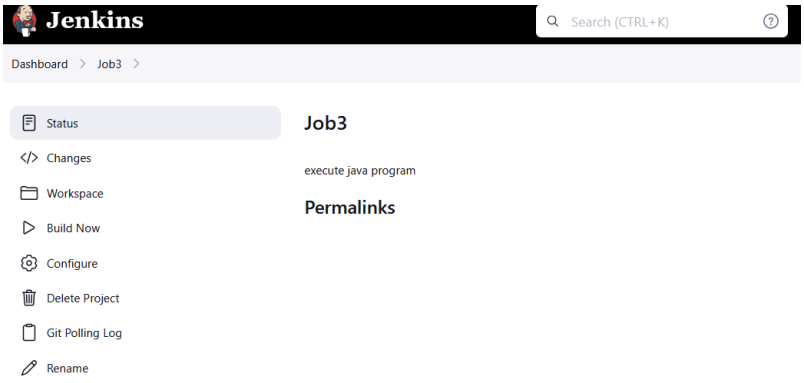
In build triggers option, select POLL SCM and schedule it as H/5 * * * *



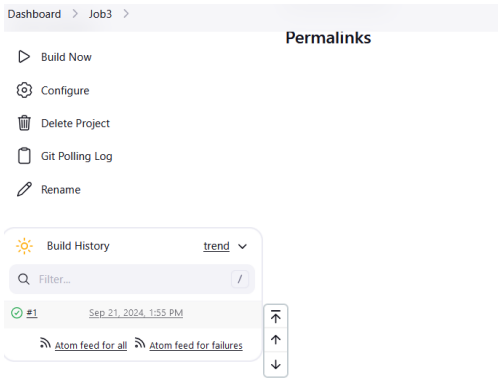
In the build steps, select Add build step and select Execute windows batch command and type the following commands to run java program



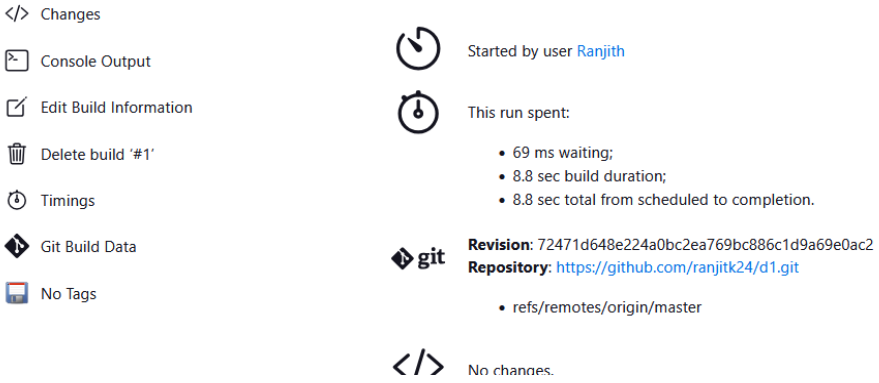
Click on Save to save the job



Now click on Build Now to build the job'



Click on the Job number from the Build History



Click on the console output to view the output of Job



Week 6: Basic Docker Commands for Content Management

Aim:

To learn how to use simple Docker commands to manage applications inside containers.

Docker is the containerization platform that is used to package your application and all its dependencies together in the form of containers to make sure that your application works seamlessly in any environment which can be developed or tested or in production

Docker is a tool designed to make it easier **to create, deploy, and run applications by using containers.**

Containerization is a **software deployment process** that bundles an application's code with all the files and libraries it needs to run on any infrastructure.

Key Components of Docker

- **Docker Engine:** It is a core part of docker, that handles the creation and management of containers.
- **Container:**A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another
- **Docker Image:** It is a read-only template that is used for creating containers, containing the application code and dependencies. It includes everything needed to run an application: ***code, runtime, system tools, system libraries and settings***.
- **Docker Hub:** It is a cloud-based repository that is used for finding and sharing the container images.
- **Docker file**: It is a script that containing instructions to build a docker image.
- **Docker Registry**: It is a storage distribution system for docker images, where you can store the images in both public and private modes